# SPAM MESSAGE DETECTOR

**A Mini Project Report**

**A report submitted**
In partial fulfilment of the requirements for the Degree of
**Bachelor of Technology**
In
**Information Technology**
Semester – V

Subject:
**Programming with Python (202044504)**

By
**Harshita V Rasania (12402080601042)**

**Department of Information Technology**
A D Patel Institute of Technology
New Vallabh Vidyanagar, Vitthal Udyognagar – 388121
Anand (Gujarat), India

# CERTIFICATE

This is to certify that the mini project work embodied in this report entitled **“Spam Message Detector”** was carried out by **Harshita V Rasania (12402080601042)** at A D Patel Institute of Technology for partial fulfilment of the requirements of the Bachelor of Technology degree awarded by Charutar Vidya Mandal (CVM) University. This project work has been carried out under the guidance and supervision of the faculty guide and is to the satisfaction of the department.

Date: _____
Place: New Vallabh Vidyanagar

| Lab Teacher | Head of Department |
|---|---|
| Nayan Mali<br>Assistant Professor | Dr. Priyang Bhatt<br>Associate Professor & Head |

# ACKNOWLEDGEMENT

I would like to express my sincere gratitude to A D Patel Institute of Technology and the Department of Information Technology for providing the opportunity and resources to undertake this mini project, “Spam Message Detector.”

I am thankful to my faculty guide and lab teacher for their guidance, suggestions, and support throughout this project. Their feedback helped me understand Python strings, regular expressions, functions, data structures, validation, and file handling.

I also thank my classmates and everyone who provided assistance and encouragement. Finally, I am grateful to my family for their continuous support.

**Harshita V Rasania**
(12402080601042)

# TABLE OF CONTENTS


| Section | Topic |
|---|---|
| 1 | Introduction and Problem Statement |
| 2 | Objectives and Scope |
| 3 | Functional and Technical Requirements |
| 4 | System Architecture and Folder Structure |
| 5 | Python Concepts and Technologies Used |
| 6 | Message Detection and Rule Management |
| 7 | File Handling and Data Persistence |
| 8 | User Interface Design |
| 9 | Validation and Error Handling |
| 10 | Testing and Test Cases |
| 11 | Setup and Run Instructions |
| 12 | Limitations and Future Improvements |
| 13 | Conclusion |
| — | Abstract and References |

# LIST OF TABLES

Table 1: Functional requirements
Table 2: Technical requirements
Table 3: Application modules and responsibilities
Table 4: Detection rules
Table 5: Record fields
Table 6: User menu controls
Table 7: Validation checks
Table 8: Manual test cases
Table 9: Setup commands

# LIST OF FIGURES

Figure 1: Application architecture and data flow
Figure 2: Console interface wireframe

# ABSTRACT

Spam Message Detector is a Python mini project that classifies user-entered text as SPAM or NOT SPAM using string processing and regular expressions. It checks messages against a predefined collection of suspicious patterns, reports the matching rules, and saves results in a local JSON history file. The application provides a simple console menu to check messages and review earlier detections. This rule-based academic demonstration does not guarantee accurate identification of all spam.


**Keywords:** Python, Spam Detection, Strings, Regular Expressions, re, JSON, File Handling.

# 1. INTRODUCTION AND PROBLEM STATEMENT

## 1.1 Background

Unwanted messages often contain promotional claims, urgent calls to action, or suspicious links. A basic detector can search for such text patterns and flag messages for review.

## 1.2 Problem Statement

Develop a Python application that accepts a text message, checks it against predefined regular-expression rules, displays a classification, and stores the result for later review.

## 1.3 Motivation

The project combines Python string methods, pattern matching, lists, dictionaries, functions, and persistent file storage in a practical example.

# 2. PROJECT OBJECTIVES AND SCOPE

## 2.1 Objectives

• Accept messages from the user.
• Detect suspicious words, phrases, and links with regular expressions.
• Classify a message based on a transparent rule threshold.
• Display which rules matched.
• Store detection records in a JSON file.
• Allow the user to view previous records.
• Validate empty input and handle file errors.

## 2.2 Scope

In scope: console input, regex-based pattern detection, basic classification, history display, and local JSON persistence. Out of scope: machine learning, real-time SMS/email integration, phishing URL verification, and guaranteed spam identification.

# 3. FUNCTIONAL AND TECHNICAL REQUIREMENTS

## 3.1 Functional Requirements

| ID | Requirement |
|---|---|
| FR1 | Accept a message from the user. |
| FR2 | Search the message using predefined regex rules. |
| FR3 | Display SPAM or NOT SPAM and matched rules. |
| FR4 | Save each successful check to JSON history. |
| FR5 | Load existing history at startup. |
| FR6 | Display saved detection history. |
| FR7 | Allow the user to exit the application. |

## 3.2 Technical Requirements

| Aspect | Technology / requirement |
|---|---|
| Language | Python 3 |
| Interface | Console input/output |
| Libraries | re, json, os, datetime (standard library) |
| Data structures | String, list, dictionary |
| Storage | spam_history.json |
| OS | Windows, macOS, Linux |

## 3.3 Non-Functional Requirements

Usability: numbered menu and clear feedback. Reliability: invalid choices, empty messages, missing history files, and JSON decoding errors are handled. Persistence: history remains available between sessions. Portability: no third-party packages are required.

# 4. SYSTEM ARCHITECTURE AND FOLDER STRUCTURE

## 4.1 Architecture Overview

| Module | Responsibility |
|---|---|
| Input/menu | Collects a message and offers history/exit choices. |
| Detection | Uses re.search() and case-insensitive patterns. |
| Classification | Counts distinct matching rules; two or more means SPAM. |
| Storage | Loads and saves JSON detection records. |

## 4.2 Data Flow

User message → empty-input validation → regex matching → classification → result display → JSON history update.

## 4.3 Folder Structure

```
Spam_Message_Detector/
 |-- spam_message_detector.py
 |-- spam_history.json
 |-- README.md
```

# 5. PYTHON CONCEPTS AND TECHNOLOGIES USED

## 5.1 Strings

The input message is a Python string. strip() removes leading and trailing whitespace; string values are passed to the regex engine.

```
message = input('Enter message: ').strip()
```

## 5.2 Regular Expressions

The re module finds patterns such as suspicious keywords and website links. Word boundaries reduce accidental matches inside larger words; re.IGNORECASE matches upper- and lower-case letters.

```
re.search(r'\b(?:winner|prize|lottery)\b', message, re.IGNORECASE)
```

## 5.3 Lists and Dictionaries

PATTERNS is a dictionary mapping rule names to regex expressions. A list stores matched rule names, and a list of dictionaries stores historical results.

## 5.4 Functions and Modules

| Function | Purpose |
|---|---|
| detect_spam() | Returns the classification and matching rule names. |
| load_history() | Loads JSON records or returns an empty list. |
| save_history() | Writes history to a JSON file. |
| main() | Runs the numbered console menu. |

# 6. MESSAGE DETECTION AND RULE MANAGEMENT

## 6.1 Detection Rules

| Rule | Example text detected |
|---|---|
| Prize or reward | won, winner, prize, lottery, gift |
| Urgent language | urgent, act now, limited time |
| Money offer | free money, cash bonus, earn money |
| Suspicious action | click here, verify your account, send otp |
| Promotional wording | exclusive offer, buy now, discount |
| Web link | http://..., https://..., www.... |

## 6.2 Classification Logic

Each rule contributes at most one match to the score. If two or more distinct rules match, the message is classified as SPAM. Otherwise, it is classified as NOT SPAM. This is a heuristic demonstration, not a statistical probability.

```
score = len(matches)
result = 'SPAM' if score >= 2 else 'NOT SPAM'
```

## 6.3 Example

“Congratulations! You won a prize. Click here https://example.com to claim now” matches prize or reward, suspicious action, and web link. The output is SPAM. “Please bring your notebook tomorrow” matches no rules and produces NOT SPAM.

# 7. FILE HANDLING AND DATA PERSISTENCE

## 7.1 Purpose

Detection records are stored in spam_history.json so the user can review earlier checks after restarting the program.

## 7.2 Saving and Loading

```
with open(FILE, 'w', encoding='utf-8') as file:
    json.dump(history, file, indent=4, ensure_ascii=False)
```

load_history() checks whether the file exists and uses json.load() to read a list of records. Missing, unreadable, or malformed history is handled by returning an empty list.

## 7.3 Record Structure

| Field | Meaning |
|---|---|
| date | Local date and time of detection. |
| message | Original user-entered text. |
| result | SPAM or NOT SPAM. |
| matched_rules | List of matching rule names. |

## 7.4 Example JSON

```
[
  {
    "date": "2026-10-26 11:25:00",
    "message": "Please bring your notebook tomorrow",
    "result": "NOT SPAM",
    "matched_rules": []
  }
]
```

# 8. USER INTERFACE DESIGN

## 8.1 Console Menu

The application uses a simple text-based interface rather than a graphical window. The menu offers Check Message, View History, and Exit.

```
===== SPAM MESSAGE DETECTOR =====
1. Check Message
2. View History
3. Exit
Enter choice:
```

## 8.2 Menu Controls

| Choice | Purpose |
|---|---|
| 1 | Accept and analyse a message, then save its result. |
| 2 | Display saved results in order. |
| 3 | End the program. |

## 8.3 User Interaction Flow

Launch → load history → choose action → enter message → validate → detect patterns → display result → save history → return to menu.

# 9. VALIDATION AND ERROR HANDLING

| Condition | Handling |
|---|---|
| Empty message | Print a warning; do not classify or save. |
| Invalid menu choice | Print Invalid choice. |
| Missing history file | Start with an empty list. |
| Malformed JSON | Start with an empty list. |
| File write error | Display the error and do not claim that history was saved. |

# 10. TESTING AND TEST CASES

| ID | Input / action | Expected result |
|---|---|---|
| T1 | Prize plus click-here link | SPAM |
| T2 | Normal reminder | NOT SPAM |
| T3 | Only one matched rule | NOT SPAM |
| T4 | Empty message | Warning |
| T5 | Uppercase suspicious text | Case-insensitive matching |
| T6 | View history after checks | Saved records shown |
| T7 | Restart application | Previous JSON history loaded |
| T8 | Invalid menu choice | Error message; menu repeats |

## 10.1 Expected Outcome

The detector correctly applies its predefined rules, reports matching categories, persists successful checks, and handles invalid input. False positives and false negatives remain possible.

# 11. SETUP AND RUN INSTRUCTIONS

## 11.1 Prerequisites

Install Python 3. No additional Python packages are required.

## 11.2 Execution Steps

Save the source code as spam_message_detector.py, open a terminal in its folder, and execute:

```
python spam_message_detector.py
python3 spam_message_detector.py
```

| Command | Purpose |
|---|---|
| python spam_message_detector.py | Run where python is Python 3. |
| python3 spam_message_detector.py | Run on macOS/Linux if required. |

# 12. LIMITATIONS AND FUTURE IMPROVEMENTS

## 12.1 Limitations

- A rule-based detector cannot understand full message context.
- A message with one suspicious pattern may be classified NOT SPAM.
- Legitimate messages may match multiple rules.
- The system does not inspect links or attachments.
- JSON history is stored locally and is not encrypted.

## 12.2 Future Improvements

- Add configurable patterns and scoring weights.
- Support CSV/text-file batch checking.
- Introduce a Tkinter interface.
- Add machine-learning classification with an evaluated dataset.
- Allow deletion and export of history.
- Add privacy controls and safer local storage.

# 13. CONCLUSION

The Spam Message Detector mini project demonstrates practical use of Python strings, regular expressions, functions, lists, dictionaries, exception handling, and JSON file handling. It provides a transparent introductory approach to message screening, with a simple menu and saved detection history. More advanced detection would require richer signals and systematic evaluation.

# REFERENCES

[1] Python Software Foundation, Python 3 Documentation. https://docs.python.org/3/

[2] Python Software Foundation, re — Regular expression operations. https://docs.python.org/3/library/re.html

[3] Python Software Foundation, json — JSON encoder and decoder. https://docs.python.org/3/library/json.html

[4] Python Software Foundation, Built-in Types. https://docs.python.org/3/library/stdtypes.html

[5] Python Software Foundation, Reading and Writing Files. https://docs.python.org/3/tutorial/inputoutput.html

**Harshita V Rasania | 12402080601042 | IT A**